

2026학년도 전기 졸업과제 중간보고서



카오스 엔지니어링 자동화 및 모니터링 플랫폼

분과명: 네트워크/통신(C)
팀명: 카엔링(KuardianAngelRing)

202055579 이시웅 (silverttthin@pusan.ac.kr)
202155543 김태운 (animoto1@pusan.ac.kr)
202155572 양준영 (y.jun0@pusan.ac.kr)

지도교수: 김태운 교수님

부산대학교 정보컴퓨터공학부

1. 목표.....	3
1.1 배경 및 필요성.....	3
1.2 최종 목표.....	3
1.3 세부 목표.....	3
2. 요구조건 및 제약사항 분석에 대한 수정사항	4
3. 설계 상세화 및 변경 내역.....	5
3.1 전체 시스템 구조.....	5
3.2 클라우드 실험 환경.....	6
3.3 GitOps 빌드, 배포 파이프라인.....	7
3.4 관측 아키텍처.....	8
3.5 카오스 테스트 실행.....	8
3.6 AI 루프 설계.....	9
3.7 DB 테이블 설계.....	10
4. 갱신된 과제 추진 계획.....	11
5. 구성원별 진척도.....	12
5.1 김태윤 — 클라우드 실험 환경과 플랫폼.....	12
5.2 이시웅 — 온프레미스 실험 환경 구축 및 AI 루프.....	12
5.3 양준영 — 온프레미스 환경 운영, AI 루프 및 대시보드 통합.....	13
6. 보고 시점까지의 수행 내용 및 중간 결과.....	13
6.1 클라우드 인프라 자동 구축과 정리.....	13
6.2 GitOps 빌드, 배포 파이프라인 실패포 검증.....	14
6.3 chaoslab 대시보드 구현.....	17
6.4 카오스 주입 기능 구현.....	18
6.5 온프레미스 환경 수행 내용.....	19
6.6 AI 루프 수행 내용.....	20
7. 결론 및 기대효과.....	21
8. 참고문헌.....	22

1. 목표

1.1 배경 및 필요성

Kubernetes 환경에서 흔히 사용되는 마이크로서비스 아키텍처(MSA)는 서비스 간 의존성이 복잡해질수록 작은 장애가 전체 시스템 장애로 확산될 위험을 안고 있다[1]. 특정 Pod의 종료나 서비스 간 네트워크 지연 같은 국소 장애가 사용자 경험 전체의 저하로 이어질 수 있으며, 이런 약점을 운영 사고가 나기 전에 미리 찾아내기 위해 의도적으로 장애를 주입하고 회복 과정을 관찰하는 방법론이 카오스 엔지니어링이다[2][3].

그러나 Chaos Mesh, LitmusChaos 같은 기존 도구는 장애 주입의 실행만 자동화할 뿐이다. 장애 가설 수립, 결과 해석, 설정 개선은 여전히 Kubernetes와 관측성 도구에 능숙한 전문가의 몫이다. 이 진입장벽 때문에 카오스 엔지니어링은 대형 기술 기업 밖으로 널리 확산되지 못하고 있다.

본 과제는 이 문제를 해결하기 위해 LLM 기반 에이전트와 관측성 도구를 결합하여, 장애 주입부터 관측, 분석, 개선, 재검증까지 이어지는 카오스 엔지니어링 전 과정을 자동화하는 플랫폼을 개발하고자 한다.

1.2 최종 목표

본 과제의 최종 목표는 클라우드(AWS EKS)와 온프레미스(라즈베리파이 기반 k3s) 환경의 마이크로서비스 시스템을 대상으로, 장애 주입 → 관측 → 분석 → 개선 → 재검증의 루프를 자동화하고 그 결과를 대시보드와 보고서로 제공하는 것이다. 회복탄력성은 R 지수라는 단일 정량 지표로 수치화하여 개선 전후를 객관적으로 비교한다.

1.3 세부 목표

착수 시점의 세부 목표를 현재 확정된 설계 기준으로 갱신하면 다음과 같다.

첫째, 사용자가 Chaos Mesh YAML을 직접 작성하지 않아도 되도록 웹 대시보드에서 장애 실험을 설계하고 주입한다. 실험 유형은 네트워크 지연, Pod 강제 종료, CPU 부하 3종을 우선 구현했으며, 파라미터는 허용 범위 검증을 거친 뒤에만 클러스터에 적용된다.

둘째, 장애 주입 전후의 시스템 상태를 Prometheus(메트릭), Loki와 Promtail(로그), Istio 사이드카(서비스 간 트래픽)로 정량 수집하고 웹 대시보드에 실시간 표시한다.

셋째, 수집 데이터를 바탕으로 회복탄력성을 R 지수로 수치화한다. 산식과 구성 요소 정의는 3.6절에 기술한다.

넷째, LLM 에이전트가 실험 결과를 해석하고 Istio 트래픽 정책 파라미터(timeout, retry, circuitBreaker) 중심의 개선안을 도출한다. 근거 없는 적용을 막기 위해 파라미터별 허용 범위를

정의하고 적용 전 검증한다.

다섯째, 개선안 적용 후 동일 장애 시나리오를 재실행하여 **R** 지수가 실제로 향상되었는지 검증하고, 반복 이력과 최종 요약 보고서를 사용자에게 제공한다.

2. 요구조건 및 제약사항 분석에 대한 수정사항

착수보고서의 요구조건과 제약사항 분석은 대부분 유지하되, 착수 이후 검증과 설계 확정을 거치며 다음 항목이 수정되었다.

실험 대상(**SUT**)은 **Online Boutique**로 확정했다. 후보로 검토하던 공개 예제 중 **Google의 Online Boutique**(11개 마이크로서비스)를 표준 실험 대상으로 확정하고 클러스터 생성 시 자동 배포되도록 구성했다. 나아가 사용자 애플리케이션을 **GitHub** 레포지토리 주소만으로 등록하면 빌드·배포까지 자동 처리되는 파이프라인을 구현해 실험 대상의 폭을 넓혔다(3.3절).

자동 개선 범위는 **Istio** 트래픽 정책 파라미터 중심으로 좁혔다. 개선 대상이 넓을수록 **LLM** 변경안의 안전성 검증이 어려워지므로, 대상을 **Istio**의 **timeout**, **retry**, **circuitBreaker** 파라미터 중심으로 축소하고 **Kubernetes replicas**와 **HPA**는 후보로만 유지했다. 대신 파라미터별 허용 범위를 사전 정의해 검증을 통과한 개선안만 적용되도록 안전장치를 강화했다.

LLM 에이전트는 **LangGraph** 기반 워크플로우형 구조로 확정했다. 카오스 엔지니어링은 가설 수립→실험→분석→개선의 순서가 고정된 작업이라, 매 단계 행동을 모델이 정하는 **ReAct**형 대신 단계별 입출력이 명확한 워크플로우형이 자연스럽고 역할 분담과 디버깅에도 유리하다고 판단했다(3.6절). **LLM** 모델은 구현 과정에서 성능과 비용을 비교해 결정한다.

관측 스택 구성을 확정하고 분산 트레이싱은 제외했다. **Prometheus**, **Loki**, **Promtail**, **Istio** 사이드카를 채택해 설치, 검증을 마쳤다. **Jaeger** 기반 분산 트레이싱은 **R** 지수 산출에 필요한 메트릭이 **Prometheus**와 **Istio** 사이드카만으로 확보된다고 판단해 제외했다.

클라우드 비용 통제 방식은 검증을 완료했다. **up.sh** 한 번으로 전체 환경이 약 20–25분에 구축되고 **down.sh**로 모든 리소스가 삭제되어 유휴 비용 0원임을 반복 확인했다(6.1절).

LLM 생성물 검증 설계는 유지하되 검증 로직을 선구현했다. **LLM** 생성물의 형식·대상·허용 범위를 적용 전 확인하는 설계는 유지하되, 그 검증 계층을 대시보드에 먼저 구현했다. **AI** 루프 도입 시 사람이 만든 실험이든 **LLM**이 만든 실험이든 동일한 검증을 통과하게 된다.

데이터 저장소는 **SQLite**에서 **Supabase**(관리형 **PostgreSQL**)로 전환을 확정했다. 실험 환경을 **up.sh/down.sh**로 생성·파괴할 때마다 **EC2** 로컬 **SQLite**의 앱 등록 정보와 실험 이력이 함께 유실되는 문제가 있어, 데이터 저장소를 클러스터 외부의 관리형 **PostgreSQL**(**Supabase**)로 분리했다. 환경을 재생성해도 이력이 보존되어 반복 실험 간 **R** 지수 비교가 가능하다(3.7절).

온프레미스 환경 관련 수정사항은 다음과 같다. 실험 대상은 Sock Shop의 ARM64 비호환으로 다른 예제로 변경했고, ChaosEater[4]의 kind 기반 구성 대신 라즈베리파이 3대(마스터 1, 워커 2)의 실물 노드 k3s[5]로 환경을 확정해 클러스터 구성을 완료했다(6.5절). 스마트 플러그 기반 물리 전원 장애 시나리오는 1차 범위에서 제외했다.

3. 설계 상세화 및 변경 내역

3.1 전체 시스템 구조

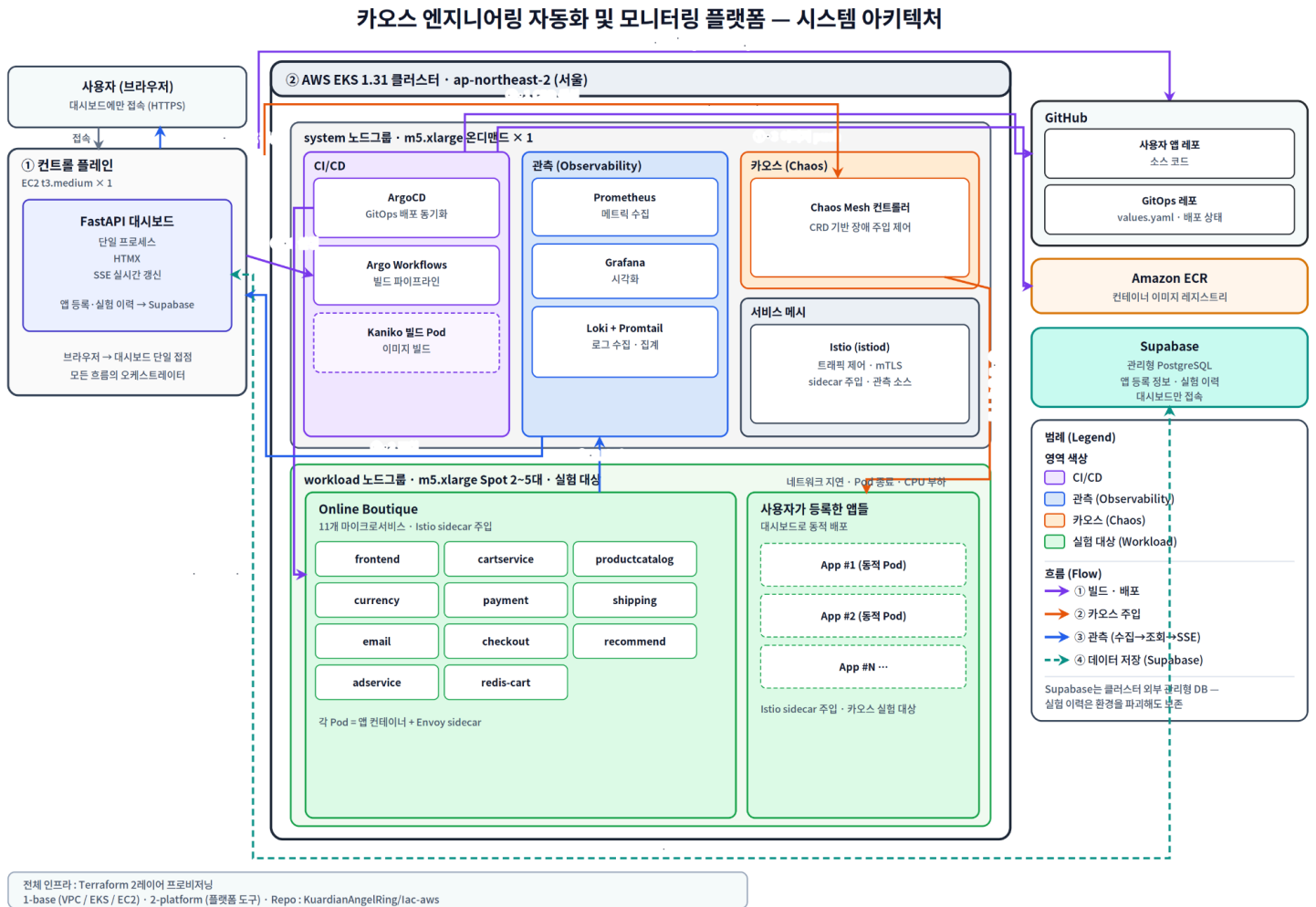


그림 1. 전체 시스템 구조

그림 1은 전체 시스템 구조다. 시스템은 인프라(**lac-aws**), 컨트롤 플레인(**chaoslab**), 실험 대상(**SUT**)이라는 세 관심사를 3개의 레포지토리로 분리하여 각자 독립적으로 변경하고 검증할 수 있게 했다. lac-aws는 Terraform 인프라 코드와 GitOps 배포 정의를, chaoslab은 대시보드와 AI 루프를 담당한다.

실행 구조는 의도적으로 단순하게 설계했다. EC2 한 대의 단일 FastAPI 프로세스가 컨트롤 플레인을 맡아 카오스 주입과 메트릭·로그 수집, 시각화를 수행하고, AI 루프도 같은 프로세스의 백그라운드 태스크로 동작한다. 데이터 저장소는 환경 파괴와 무관하게 실험 이력을 보존하기 위해 관리형 PostgreSQL(Supabase) 하나만 둔다. 팀 내부 사용자 수준의 트래픽에서 워커 프로세스나 메시지 큐 같은 분산 구성은 운영 복잡도만 높인다고 판단했고, 절약한 복잡도 예산은 파이프라인 자동화와 검증에 투자했다.

3.2 클라우드 실험 환경

클라우드 실험 환경은 콘솔 수작업 없이 Terraform 코드만으로 전체를 재현할 수 있게 만들었고, 기반 인프라(1-base)와 플랫폼 도구(2-platform)의 2레이어로 분리했다. 한 번 만들면 거의 바뀌지 않는 기반 인프라와 달리 플랫폼 도구는 버전과 설정 변경이 잦아, EKS 클러스터를 재생성하지 않고도 플랫폼 구성만 따로 변경하거나 재설치할 수 있게 한 것이다.

1-base 레이어는 VPC(NAT Gateway 1개), EKS 1.31 클러스터, 컨트롤 플레인용 EC2(t3.medium)를 생성한다. 노드그룹은 플랫폼 컴포넌트 전용 system(온디맨드 1대, taint 격리)과 실험 대상 전용 workload(Spot, 기본 2대·최대 5대)의 두 개로 나뉘었다. 장애를 주입하는 실험이 관측 스택과 같은 노드에서 돌면 관측 도구까지 함께 무너뜨려 측정 자체가 불가능해질 수 있으므로, 실험의 영향 범위를 workload 노드로 한정해 심판(관측 스택)과 선수(실험 대상)를 분리한 것이다. ECR 접근에는 액세스 키를 노드에 저장하지 않는 IRSA(IAM Roles for Service Accounts)를 적용해 자격증명 유출 위험을 제거했다.

2-platform 레이어는 Kubernetes 위에서 동작하는 플랫폼 도구 전체를 Helm으로 설치한다. 전체 기술 스택은 다음 표 1과 같다.

구분	기술	역할
클러스터	AWS EKS 1.31	실험용 Kubernetes 클러스터 (system/workload 노드그룹 분리)
인프라 코드화	Terraform 1.6 이상	전체 환경을 코드로 정의 (1-base/2-platform 2레이어)
빌드(CI)	Argo Workflows, Kaniko	클러스터 내부 이미지 빌드와 ECR 업로드
배포(CD)	ArgoCD	Git 배포 정의를 클러스터에 자동 동기화
실험 대상 앱	Online Boutique	표준 실험 대상 (11개 마이크로서비스 공개 예제)
카오스 주입	Chaos Mesh 2.8.2	장애 주입 실행기 (대시보드가 CRD로 직접 제어)
서비스 메시	Istio 1.29.2	트래픽 메트릭 제공, AI 개선 파라미터 적용 지점
메트릭	Prometheus, Grafana	메트릭 수집(15초 주기, 7일 보존)과 시각화

구분	기술	역할
로그	Loki, Promtail	로그 수집과 통합 조회
대시보드	FastAPI, HTMX	웹 대시보드 (단일 프로세스, SSE 실시간 갱신)
데이터 저장	Supabase (관리형 PostgreSQL)	앱 등록 정보와 실험 이력 저장
AI 루프	LangGraph, LLM	실험 결과 분석과 개선안 생성

표 1. 클라우드 실험 환경 기술 스택

3.3 GitOps 빌드, 배포 파이프라인

사용자 애플리케이션의 빌드부터 배포까지를 **GitOps** 원칙으로 자동화했다. **GitOps**란 클러스터에 무엇이 배포되어야 하는지를 **Git** 레포지토리에 선언해 두고, 클러스터의 실제 상태를 항상 **Git**과 일치시키는 방식이다[6]. 배포의 단일 진실 원천이 **Git**이 되므로 모든 배포 변경이 커밋 이력으로 남고, 클러스터를 삭제했다가 재생성해도 **Git** 기준으로 동일한 상태가 자동 복원된다.

그림 2. 새 앱 등록 폼의 환경변수 입력 단계

파이프라인은 사용자가 대시보드에서 **GitHub** 레포지토리 주소, 프레임워크, 헬스체크 경로, 환경변수와 시크릿을 입력해 앱을 등록하는 것으로 시작한다(그림 2). 설정값은 이미지에 굵지 않고 배포 시점에 주입하는 **12-factor** 원칙을 따라, 평문 설정은 **GitOps** 레포지토리의 **values.yaml**에 커밋하고 비밀값은 공개 레포지토리에 노출되지 않도록 클러스터의 **Kubernetes**

Secret으로 직접 생성해 분리 주입한다.

등록 후 빌드 버튼을 누르면 클러스터 내부의 **Argo Workflows**가 레포지토리를 **clone**하고 **Kaniko**로 이미지를 빌드해 **ECR**에 **push**한다. 이미지 태그는 커밋 **SHA**를 그대로 사용해 어떤 코드로 만든 이미지인지 항상 추적되며, 레포지토리에 **Dockerfile**이 없으면 프레임워크별 템플릿을 자동 주입하므로 사용자는 **Dockerfile**을 몰라도 된다.

빌드가 끝나면 대시보드가 새 이미지 태그를 **GitOps** 레포지토리의 **values.yaml**에 자동 커밋하고, **ArgoCD**가 이를 감지해 공용 **Helm** 차트(**generic-app**)를 렌더링하여 클러스터에 동기화한다. **selfHeal** 옵션으로 클러스터를 직접 수정해도 **Git** 상태로 복원되며, 앱 목록은 **App-of-Apps** 패턴으로 관리해 앱 추가와 삭제도 **Git** 커밋만으로 이루어진다.

대시보드는 전 과정의 상태를 **SSE(Server-Sent Events)**로 실시간 표시하고, 재빌드·재배포·배포 중지·빌드 중지 액션과 앱별 빌드·배포 이력 조회를 제공한다.

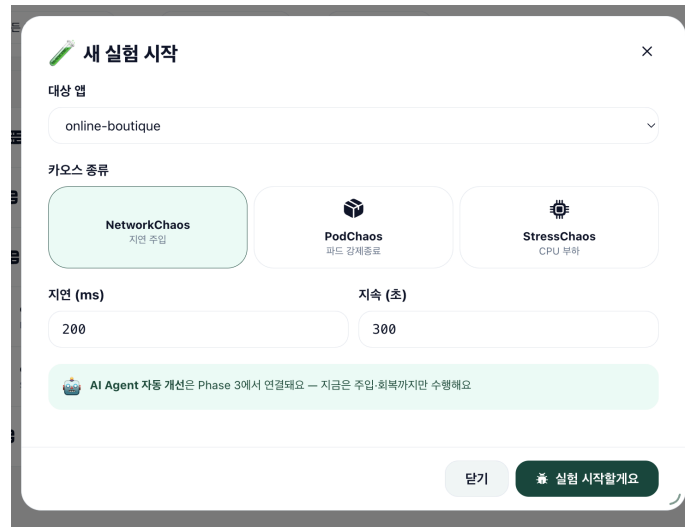
3.4 관측 아키텍처

관측 아키텍처는 메트릭과 로그 두 축으로 구성했다. 메트릭은 **kube-prometheus-stack**의 **Prometheus**가 15초 주기로 수집하고 7일간 보존하며, 여기에 **Istio** 사이드카가 노출하는 서비스 간 트래픽 메트릭(요청 수, 에러율, 레이턴시)을 함께 스크레이프한다. 사이드카가 모든 서비스 간 호출을 관찰해 주므로 애플리케이션 코드 수정 없이 어떤 앱이든 **RED** 지표(요청량, 에러, 지연)를 얻을 수 있고, 이것이 **R** 지수 산출의 핵심 기반이다.

로그는 각 노드의 **Promtail**이 수집해 **Loki**로 전송하고, **Grafana**에서 메트릭과 로그를 통합 조회한다. 모든 관측 컴포넌트는 **system** 노드그룹에 격리 배치하여 카오스 실험의 영향권 밖에 뒀다(3.2절).

3.5 카오스 테스트 실행

카오스 주입은 **Chaos Mesh**를 설치하고, 대시보드가 **Kubernetes API**의 **CRD**를 통해 직접 제어하는 구조로 구현했다. 자체 UI 대신 **CRD**를 직접 다루는 이유는, 실험의 생성과 상태와 이력을 대시보드 **DB**와 **AI** 루프가 일관되게 관리해야 하기 때문이다.



새 실험 시작

대상 앱
online-boutique

카오스 종류

NetworkChaos
지연 주입

PodChaos
파드 강제종료

StressChaos
CPU 부하

지연 (ms) 200 지속 (초) 300

AI Agent 자동 개선은 Phase 3에서 연결돼요 — 지금은 주입·회복까지만 수행해요

닫기 실험 시작할게요

그림 3. 새 카오스 실험 생성 모달

그림 3은 새 실험 모달창이다. 사용자가 대상 앱, 장애 유형, 파라미터를 입력하면 서버가 허용 범위(네트워크 지연 10–10000ms, CPU 부하 1–100%, 지속 시간 30–1800초)를 검증한 뒤 CRD를 생성해 주입하며, 범위를 벗어난 요청은 클러스터에 달기 전에 거부된다. Pod 강제 종료는 파라미터 없는 원샷 동작이다.

실험 생명주기는 자동으로 관리된다. 주입 후 백그라운드 워커가 지속 시간 동안 대기하고 회복 상태를 확인한 뒤 **CRD**를 자동 삭제해 완료 처리하므로, 실험 **CRD**가 클러스터에 잔존해 장애가 계속되는 사고를 구조적으로 방지한다. 진행 상태는 **SSE**로 실시간 갱신되고, 진행 중 실험의 중지와 동일 앱 중복 실험 차단(409)도 지원한다.

3.6 AI 루프 설계

AI 루프는 **ReAct**형 자율 에이전트가 아니라 **LangGraph** 기반 워크플로우형 에이전트 구조로 설계를 확정했다. 전체 워크플로우는 전처리→가설 생성→실험→분석→개선→보고서 생성의 6단계로 구성한다.

관측과 개선 루프는 5개 역할 에이전트가 분담한다. **Observer**가 에러율, p99 레이턴시, 평균 복구 시간(MTTR) 등 메트릭을 수집하고, **Analyst**가 원인과 병목을 분석하며, **Recommender**가 허용 범위 안에서 Istio 파라미터 개선안을 제안하고, **Executor**가 VirtualService와 DestinationRule에 적용하며, **Verifier**가 R 지수를 재측정해 개선 여부를 판정한다. 개선되지 않으면 추천 단계로 되돌아가고, 사용자가 설정한 최대 반복 횟수에서 종료한다.



그림 4. 실험 상세의 AI 루프 탭

그림 4는 이 설계를 반영해 선구현한 실험 상세의 AI 루프 탭으로, 반복별 R 지수 추이와 개선 전후 비교, 누적 LLM 비용을 한 화면에 표시한다. 에이전트는 Tool Calling으로 kubectl, Prometheus, Loki를 직접 호출해 판단 근거를 수집하고, 실험 후보는 에이전트가 생성하되 실행 대상은 사용자가 선택하는 방식으로 구성한다. 안전장치로 파라미터 허용 범위 강제, R 지수 악화 시 롤백, 누적 LLM 비용 상한을 포함했다.

구성 요소	가중치	정의
가용성(Availability)	0.4	$1 - (5xx \text{ 응답 수} / \text{전체 응답 수})$
레이턴시 점수(LatencyScore)	0.3	$\min(1, \text{기준 p99 레이턴시} / \text{현재 p99 레이턴시})$
복구 속도(RecoverySpeed)	0.3	$1 - \text{clip}(\text{MTTR} / 60\text{초}, 0, 1)$

표 2. R 지수 구성 요소와 가중치

3.7 DB 테이블 설계

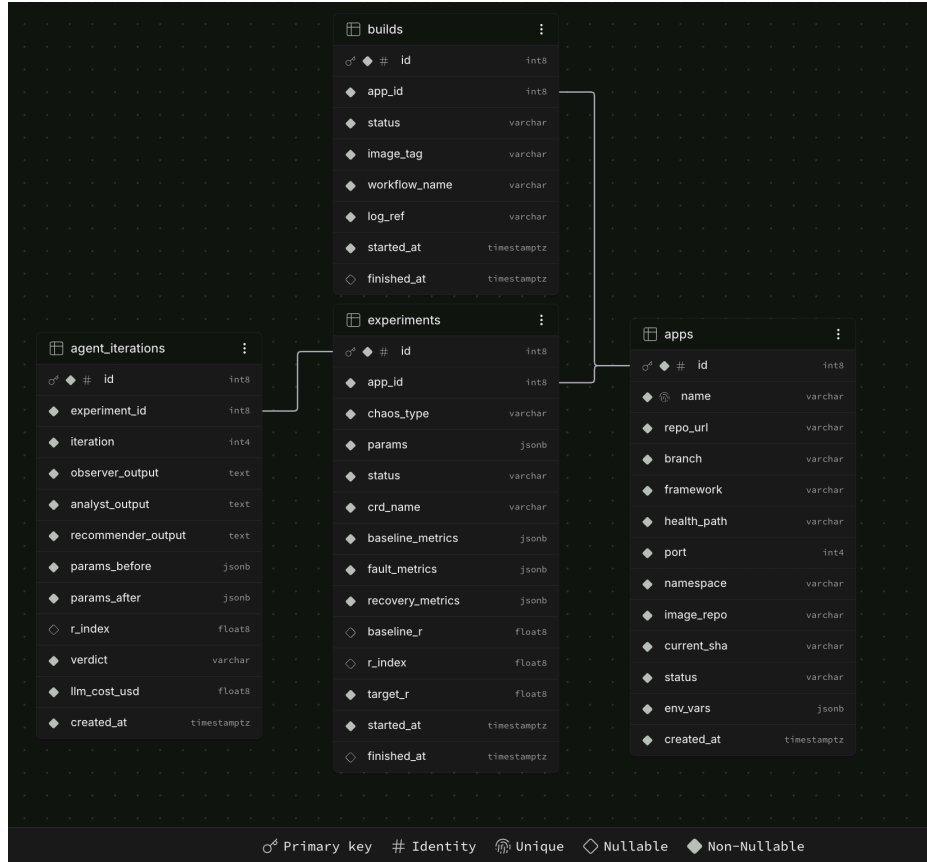


그림 5. Supabase 4개 테이블 관계도

그림 5는 Supabase에 생성한 4개 테이블의 관계도다. 클러스터와 배포 상태는 **Terraform**과 **Git**으로 언제든지 동일하게 재현되지만, 실험 이력과 **R** 지수는 그 시점의 측정값이라 한번 잃으면 재구성할 수 없다. 그래서 클러스터, **ArgoCD**, **Prometheus** 등 외부 시스템의 상태는 조회 시점에 읽어오고, **DB**에는 외부에서 재구성할 수 없는 앱 등록 정보와 실험 이력만 4개 테이블로 남긴다.

apps 테이블은 사용자가 등록한 애플리케이션의 원본 정보로, 포트·헬스체크 경로·네임스페이스·환경변수 등 배포 설정이 여기서 출발해 파이프라인 전체로 전파된다.

builds 테이블은 앱과 1:N 관계인 빌드 이력으로, **Argo Workflow** 이름과 결과 이미지 태그(커밋 SHA)를 기록해 어떤 빌드가 어떤 코드로 어떤 이미지를 만들었는지 추적한다.

experiments 테이블은 앱과 1:N 관계인 카오스 실험 이력으로, 주입된 **CRD** 이름으로 위치가 삭제 대상을 추적하고, 장애 전(**baseline**)·중(**fault**)·후(**recovery**) 구간별 메트릭 스냅샷과 **R** 지수 컬럼은 추후 **AI** 루프가 바로 사용하도록 미리 반영했다.

agent_iterations 테이블은 실험과 1:N 관계인 **AI** 루프 반복 이력으로, 회차별 에이전트 산출물과 개선 전후 **Istio** 파라미터, **R** 지수, 판정, **LLM** 비용을 남겨 루프 전체를 재구성할 수

있게 한다.

4. 갱신된 과제 추진 계획

착수보고서의 추진 일정을 보고 시점 기준으로 갱신했다. 상반기 계획 항목은 모두 완료했다. 세부 수행 내용과 증적은 6장에 기술한다. 전체 일정의 갱신 현황은 다음 표 3과 같다.

업무	기간	상태	비고
요구사항 분석 및 전체 시스템 설계	5월	완료	SUT·개선 범위·R 지수 산식 확정
클라우드 실험 환경 구축	5-6월	완료	up.sh/down.sh 자동화, 반복 검증
GitOps 빌드, 배포 파이프라인	6월	완료	Argo Workflows, Kaniko, ECR, ArgoCD 실배포 검증
Chaos Mesh 장애 주입 기능	7월	완료	3종 실험, 라이브 클러스터 검증 예정
온프레미스(k3s) 실험 환경 구축	5-8월	완료	원격 SSH 환경 구축
모니터링 실테이터 연동 (Prometheus, Loki 조회)	7월	예정	대시보드 차트 실테이터 배선
R 지수 계산 및 결과 화면	7월	예정	baseline, fault, recovery 구간 계산
LLM 기반 분석, 개선 자동화	7-8월	진행 중	구조 설계 확정, 구현 착수
워크플로우와 환경 간 연결 및 디버깅	8월	예정	
보고서 생성 기능 구현	8월	예정	
테스트, 디버깅 및 최종보고서	8월	예정	
장애 진단 골든셋 구축 (RAG)	8월	예정	검증 사례 축적, AI 루프에 제공

표 3. 갱신된 과제 추진 일정

추후 작업은 네 갈래로 구체화했다.

첫째, 모니터링 실테이터 연동이다. 지금까지 스텝 데이터로 검증한 대시보드 화면에 Prometheus와 Loki를 실배선하여, 실험 화면에 Istio 사이드카 기반 RED 지표와 Pod 가용성, 장애 구간의 로그를 실시간으로 표시한다. 실험 시각을 기준으로 장애 전(baseline), 중(fault), 후(recovery) 구간을 구분해 조회하는 것이 핵심이며, 이 구간별 데이터가 다음 단계인 R 지수 계산의 입력이 된다.

둘째, R 지수 계산과 결과 화면이다. 구간별 메트릭 스냅샷으로부터 3.6절의 산식에 따라 R 지수를 산출해 실험 상세 화면에 표시하고, 실험 이력 간 비교가 가능하게 한다.

셋째, **AI** 루프 구현이다. 3.6절에서 확정한 **LangGraph** 워크플로우를 구현해 관측, 분석, 개선, 재검증 루프를 완성한다. 이때 진단의 일관성을 확보하기 위해 장애 진단 골든셋을 함께 구축한다. **LLM**은 같은 장애에도 실행마다 다른 진단을 내놓을 수 있으므로, 검증된 장애 사례와 개선안을 골든셋으로 축적하고 **RAG**(검색 증강 생성)로 에이전트에 제공해 같은 유형의 문제는 같은 방법으로 해결하도록 유도한다. 실험이 반복될수록 진단 품질이 누적 개선되는 구조다.

넷째, 온프레미스 환경과 대시보드의 연결이다. 라즈베리파이 기반 **k3s** 클러스터를 대시보드에 연결해 클라우드와 동일한 등록, 실험, 관측 흐름을 온프레미스에서도 수행하고, 두 인프라 프로파일의 회복탄력성 특성을 비교한다.

5. 구성원별 진척도

역할 분담은 다음 표 4와 같고, 구성원별 상세 진척은 아래 절에서 서술한다.

구성원	담당
김태윤	클라우드 실험 환경, GitOps 파이프라인, 대시보드
이시웅(팀장)	온프레미스(k3s) 실험 환경, AI 루프(전처리·가설·실험)
양준영	AI 루프(분석·개선), 대시보드 통합, 온프레미스 운영

표 4. 구성원별 역할 분담

5.1 김태윤 — 클라우드 실험 환경과 플랫폼

클라우드 파트의 상반기 목표였던 실험 환경의 완전 자동화를 달성했다. 구체적으로는 세 가지 작업을 완료했다.

첫째, 실험 인프라를 코드로 만들었다. **Terraform**을 **1-base**와 **2-platform**의 2레이어로 구성하고, **up.sh** 한 번으로 전체 환경이 약 **20–25분**에 구축되며 **down.sh** 한 번으로 모든 리소스가 삭제되는 구조를 반복 검증했다. 구축 과정에서 마주친 인프라 이슈 **6건**을 전부 원인 분석부터 해결까지 마치고 레포지토리 문서(**docs/issues**)로 남겼다.

둘째, **GitOps** 빌드 배포 파이프라인을 구현하고 실제 클러스터에서 끝까지 검증했다. **Argo Workflows**와 **Kaniko**로 클러스터 내부 빌드를, **ECR**과 **ArgoCD**로 이미지 저장과 배포 동기화를 연결해, 실제 **Spring** 애플리케이션이 등록부터 **Pod** 기동까지 사람 개입 없이 배포되는 것을 확인했다(6월 2일).

셋째, 플랫폼의 얼굴인 **chaoslab** 대시보드를 구현했다. 6개 페이지 화면과 앱 등록, 빌드와 배포 제어, 환경변수와 시크릿 주입, 카오스 실험 3종의 생성부터 자동 종료까지를 구현했고, 자동화 테스트 **89건**이 통과하는 상태를 유지하고 있다. 세부 결과와 그 과정의 설계 판단은

6장에 기술한다.

5.2 이시웅 — 온프레미스 실험 환경 구축 및 AI 루프

온프레미스 파트의 상반기 목표였던 물리 노드 기반 **k3s** 실험 환경 구축을 달성했다. 라즈베리파이 3대를 마스터 1대와 워커 2대로 구성해 **k3s** 클러스터를 세우고 노드 간 연결을 검증했다. 실험 대상 배포 과정에서는 **Sock Shop**이 **ARM64** 환경에서 동작하지 않는 문제를 확인하고, **ARM64** 호환 마이크로서비스 예제와 **nginx**로 대체하여 클러스터 위에 실제로 배포되는 것까지 확인했다. 하반기에는 **Chaos Mesh**와 **k6**를 온프레미스 클러스터에 설치해 자체 작성한 에이전트 루프를 적용하고, 클라우드와 동일한 장애 주입 흐름을 확보한 뒤 대시보드와 연결해 두 인프라 프로파일의 회복탄력성을 비교한다.

5.3 양준영 — 온프레미스 환경 운영, AI 루프 및 대시보드 통합

상시 운영 중인 라즈베리파이 운영환경을 구축하고, 팀원의 온프레미스 작업을 위한 **SSH** 원격 접속 환경을 구성했다. **AI** 파트에서는 **LLM** 에이전트를 단계별 워크플로우 구조로 설계하고, 사용자가 실험 방향과 반복 범위를 조정하는 대시보드 연동 방안과 클러스터 상태·관측 데이터 조회 구조를 함께 구체화했다. 현재 전체 구조와 기능 범위 정의를 완료하고 개발에 착수한 상태이며, 세부 설계와 수행 내용은 6.6절에 기술한다.

6. 보고 시점까지의 수행 내용 및 중간 결과

6.1 클라우드 인프라 자동 구축과 정리

실험 환경의 생성과 삭제를 스크립트 하나로 재현 가능하게 만들었다. 이것이 상반기 클라우드 파트에서 가장 공들인 결과물이다. 카오스 실험은 성격상 환경을 부수고 다시 만드는 일이 잦은데, 환경 구축이 콘솔 수작업에 의존하면 실험 반복이 불가능하고, 커 둔 채 잊으면 비용이 새기 때문이다.

up.sh는 5단계로 전체 환경을 구축하며, 단계별 실효 소요 시간은 다음 표 5와 같다.

단계	내용	소요 시간(실효)
1/5	terraform apply 1-base (VPC, EKS, EC2, 노드그룹)	약 14분
2/5	로컬 kubeconfig 설정	즉시
3/5	EC2 초기화 대기 및 kubeconfig 전송	약 2분

단계	내용	소요 시간(실측)
4/5	terraform apply 2-platform (플랫폼 도구 전체) 및 App-of-Apps 등록	약 6-8분
5/5	Prometheus, Loki 포트포워딩	약 1분

표 5. up.sh 단계별 구축 소요 시간 (실측)

down.sh는 **Kubernetes**가 만든 **LoadBalancer**를 먼저 삭제한 뒤 두 레이어를 역순으로 **destroy**한다. **LoadBalancer**를 선삭제하는 이유는 **Terraform**이 모르는 리소스(**Kubernetes**가 동적으로 만든 **ELB**)가 남아 있으면 **VPC** 삭제가 실패하기 때문이다. 이 순서까지 스크립트에 담아, **EKS**, **EC2**, **VPC**, **NAT Gateway**를 포함한 모든 리소스가 제거되고 실행 후 **AWS** 비용이 0원으로 정리되는 것을 반복 확인했다.

구축 자동화가 한 번에 완성된 것은 아니다. 과정에서 발생한 이슈 6건을 해결하며 스크립트와 **Terraform** 코드를 다듬었고, 대표 사례는 다음과 같다. 가장 심각했던 것은 **Istio** 사이드카 주입 웹훅 차단 문제였다. **Online Boutique**를 포함해 사이드카 주입이 필요한 모든 **Pod**의 생성이 실패했는데, 원인을 추적해 보니 **EKS** 노드에서 **istiod** 웹훅(15017 포트)으로 가는 통신이 노드 보안그룹에 막혀 있었고, 보안그룹 규칙을 **Terraform**에 추가해 해결했다. **Loki**는 컨테이너 파일시스템이 읽기 전용이라 기동에 실패해 쓰기 경로 설정을 조정했고, **PVC**가 바인딩되지 않는 문제는 **EKS**에 **EBS CSI** 드라이버 애드온이 기본 설치되지 않는 것이 원인이라서 **1-base** 레이어에 애드온을 추가했다. 환경 삭제 시 **Chaos Mesh CRD**의 **finalizer** 때문에 네임스페이스 삭제가 멈추는 문제는 정리 절차를 확립해 **down.sh** 흐름에 반영했다. 6건 모두 증상, 원인, 해결책을 문서(docs/issues)로 남겨 같은 문제가 재발해도 즉시 대응할 수 있게 했다.

6.2 GitOps 빌드, 배포 파이프라인 실배포 검증

3.3절에서 설계한 파이프라인을 실제 클러스터에서 두 차례 끝까지 검증했다. 6월 2일 **Spring Boot** 데모 애플리케이션으로 최초 검증을 마쳤고, 보고 시점인 7월에는 실제 운영 목적으로 개발된 코드몬스터(**comon**) 백엔드 서비스를 대상으로 재검증했다. 코드몬스터 백엔드는 **MySQL**, **AWS S3**, **OAuth** 로그인 등 외부 의존성을 실제로 사용하는 **Spring Boot** 애플리케이션으로, 데모가 아닌 실서비스 수준의 코드베이스다. 대시보드에서 **GitHub** 레포지토리 주소를 입력해 등록하자 **GitOps** 레포지토리에 배포 정의가 자동 커밋되었고, 빌드 버튼 한 번에 클러스터 내부 **Kaniko** 빌드, **ECR push**, 이미지 태그 자동 커밋, **ArgoCD** 동기화, 실제 이미지 기반 **Pod** 기동까지 사람 개입 없이 이어졌다. 앱 등록에서 배포까지의 전 구간이 자동으로 연결된다는 플랫폼의 핵심 전제가 실서비스급 애플리케이션에서도 성립함을 입증한

것이다.

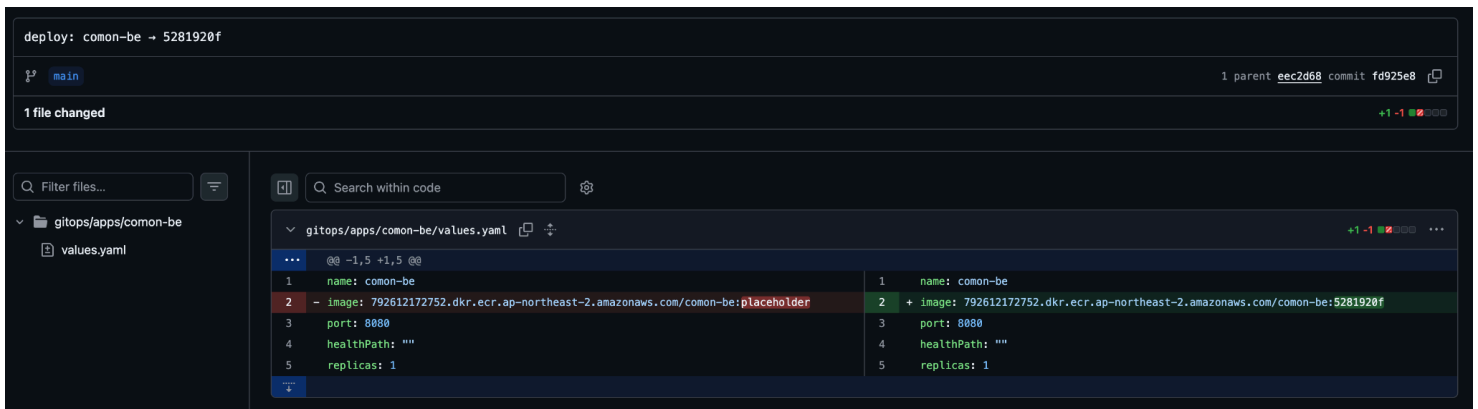


그림 6. 대시보드가 자동 생성한 GitOps 레포지토리 커밋 이력

그림 6이 이 검증의 실물 증거다. 등록 시점에 대시보드가 배포 정의를 생성한 커밋(`feat: register comon-be`)과, 빌드 완료 직후 `values.yaml`의 이미지 태그를 빌드 결과물의 커밋 **SHA**로 갱신한 커밋(`deploy: comon-be → 5281920f`)이 GitOps 레포지토리 이력에 그대로 남아 있다. 두 커밋 모두 사람이 아니라 대시보드가 만든 것이다.

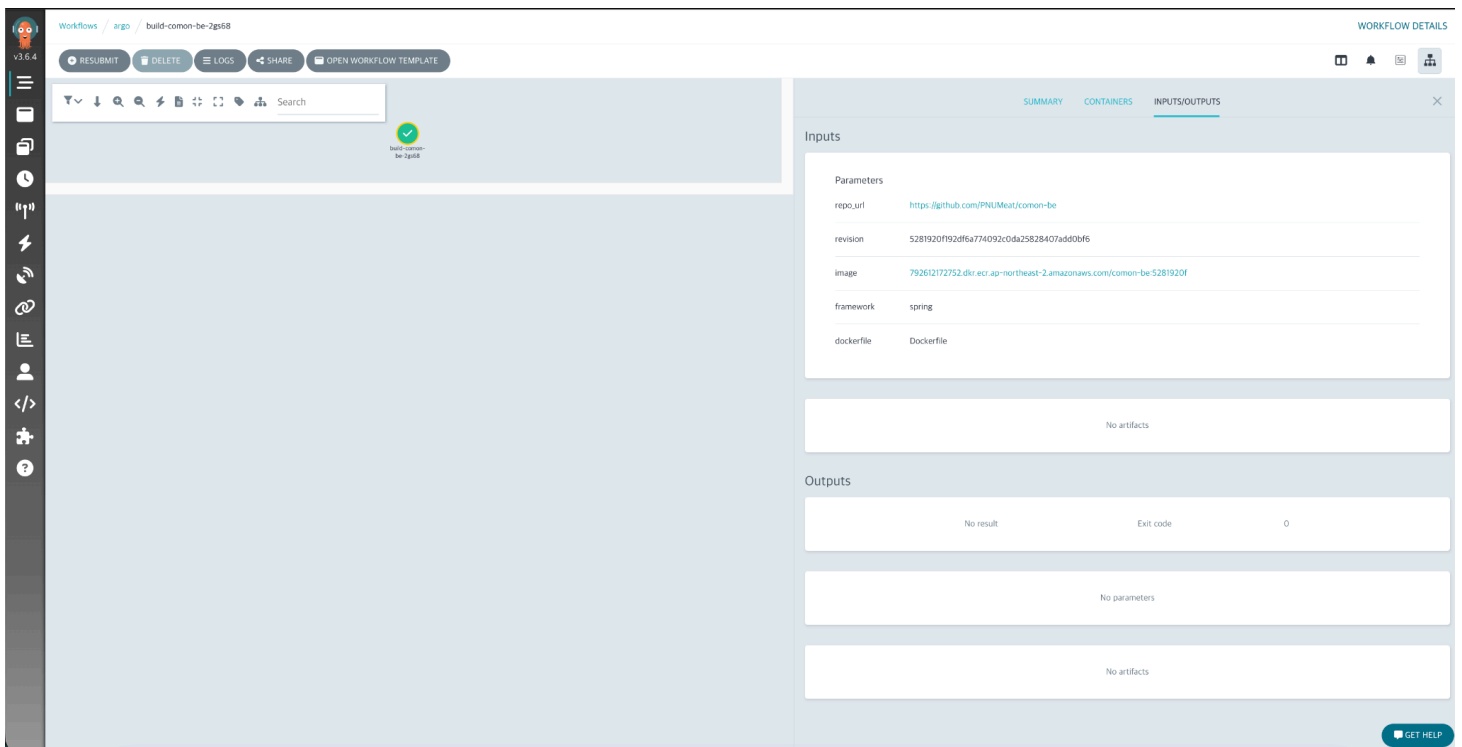


그림 7. 코드몬스터 백엔드의 Argo Workflows 빌드 완료 화면

그림 7은 코드몬스터 백엔드의 빌드 워크플로우가 클러스터 내부에서 성공적으로 완료된 모습이다. 워크플로우가 레포지토리를 clone하고 Kaniko로 이미지를 빌드해 ECR에 push하는 전 과정을 하나의 스텝으로 수행하며, 이미지 태그에는 빌드한 커밋의 SHA가 그대로 기록되어 어떤 코드로 만든 이미지인지 추적된다. 빌드는 공용 **WorkflowTemplate** 하나로 정의되어 있어, 앱이 늘어나도 레포지토리 주소와 이미지명 같은 파라미터만 바꿔 동일한 워크플로우를 재사용한다. 코드몬스터 백엔드는 레포지토리에 Dockerfile이 없는 상태로 등록되었지만, 3.3절의 프레임워크별 템플릿 자동 주입 덕분에 Spring용 Dockerfile이 빌드 시점에 주입되어 별도 준비 없이 빌드가 성립했다. ECR 인증에는 3.2절의 IRSA가 적용되어 워크플로우 Pod가 액세스 키 없이 이미지를 push한다. 빌드 상태는 대시보드의 백그라운드 위치가 **Argo Workflow**를 폴링해 DB에 기록하고 SSE로 앱 카드의 상태 배지를 실시간 갱신하며, 빌드별 상태·커밋 SHA·소요 시간·워크플로우 이름은 앱별 이력 모달에서 조회된다. 실패한 빌드도 어떤 커밋에서 어떤 워크플로우가 실행됐는지 즉시 추적할 수 있는 구조다.

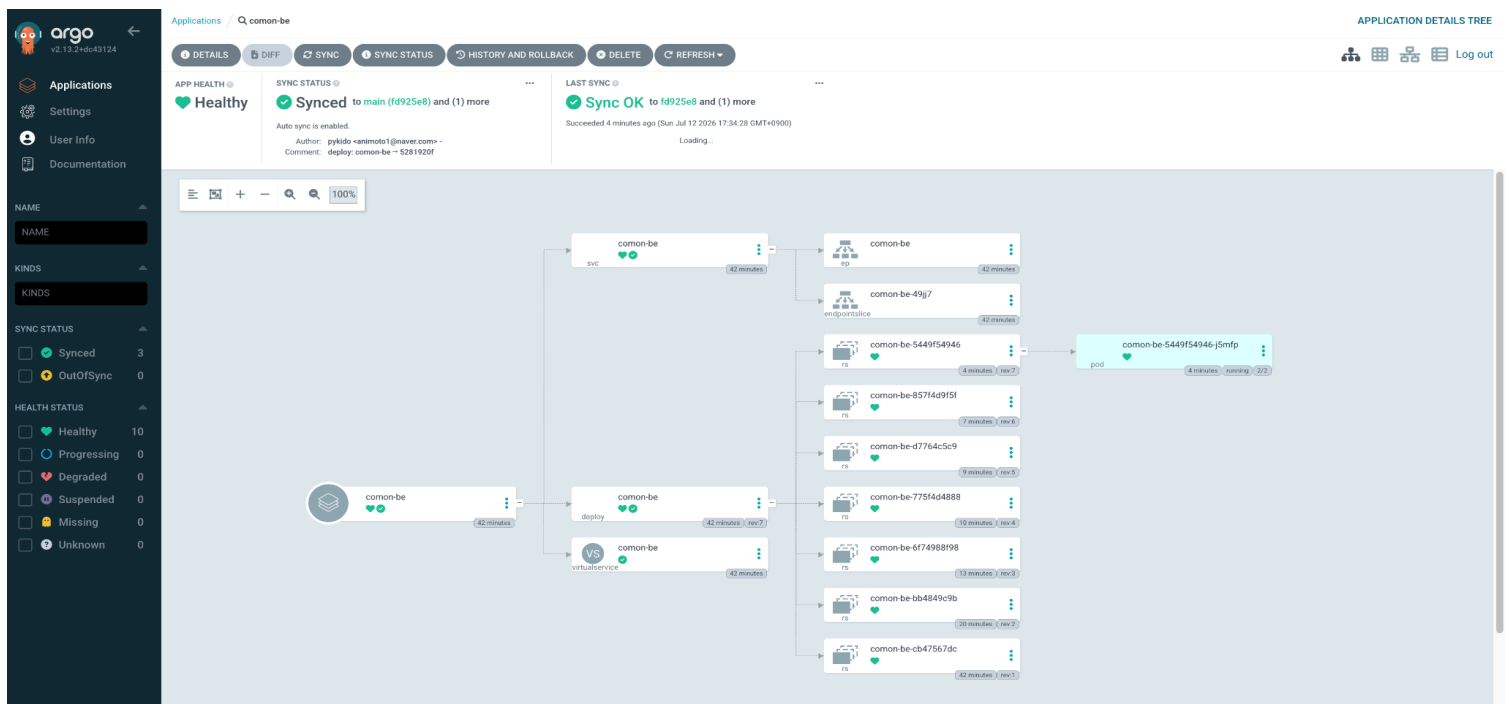


그림 8. 배포 완료 후 ArgoCD 동기화 상태 (Synced/Healthy)

그림 8은 배포 완료 후의 **ArgoCD** 상태다. 대시보드가 커밋한 이미지 태그 변경을 **ArgoCD**가 감지해 클러스터에 동기화했고, 코드몬스터 백엔드를 포함한 모든 애플리케이션이 **Git** 선언과 일치(**Synced**)하며 정상 구동(**Healthy**) 상태임을 보여준다. 화면의 각 **Application**은 GitOps 레포지토리의 앱 디렉토리와 1:1로 대응하고, 루트 **Application**이 디렉토리 전체를 감시하는

App-of-Apps 구조가 실제 클러스터에서 동작하는 모습이다. Synced는 클러스터의 리소스가 Git에 선언된 상태와 일치함을, Healthy는 그 위의 Pod가 정상 기동해 헬스체크를 통과했음을 뜻한다. 따라서 이 화면 하나가 앱 등록 커밋에서 시작된 변경이 빌드, 이미지 태그 갱신을 거쳐 실제 Pod 기동까지 사람 개입 없이 달렸다는 것을 보여주는 최종 증거가 된다.

실배포 검증 이후에는 실사용에서 필요해지는 운영 기능을 추가로 구현했다. 환경변수와 시크릿 주입은 평문은 values.yaml에, 비밀값은 클러스터 Secret에 분리 저장하도록 구현했다. 또한 재빌드, 재배포, 배포 중지, 빌드 중지 액션을 구현해 배포 운영에 필요한 제어를 대시보드 안에서 완결했다.

6.3 chaoslab 대시보드 구현

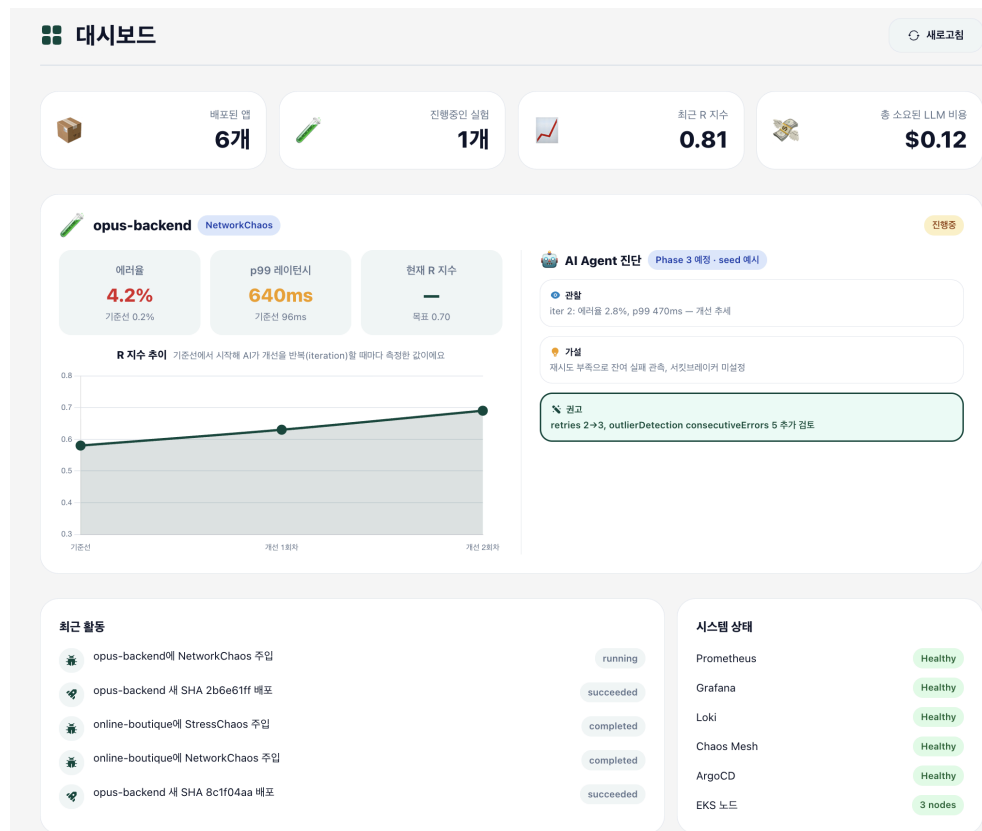


그림 9. 대시보드 개요 화면 - 진행 중 실험의 메트릭과 R 지수 추이

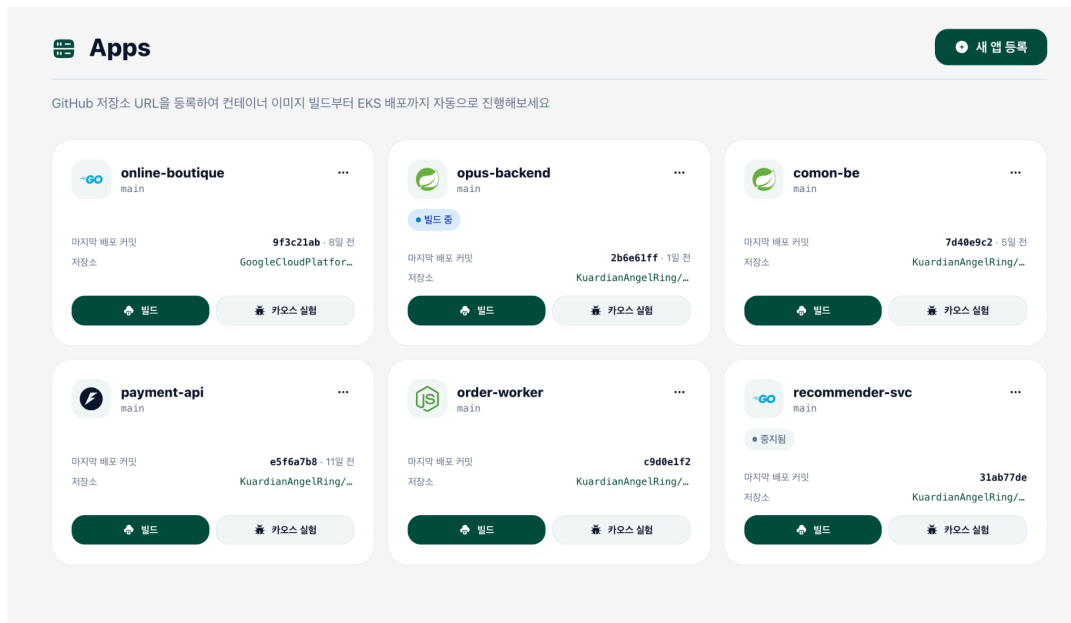


그림 10. Apps 화면 - 앱별 배포 상태와 빌드, 카오스 실험 진입점

그림 9와 그림 10은 대시보드 개요와 **Apps** 페이지다. 대시보드는 FastAPI, Jinja2, HTMX, Chart.js 스택의 단일 프로세스로 구현했으며, 대시보드(개요), Apps, 카오스 테스트, 실험 상세, EKS 인프라, 설정의 6개 페이지로 구성된다. 개요 화면은 진행 중 실험의 에러율과 p99 레이턴시, R 지수 추이를 한눈에 보여주고, Apps 화면은 앱별 배포 상태와 빌드, 카오스 실험 진입점을 제공한다.

구현에서 가장 중요한 설계 판단은 외부 시스템 접근의 추상화였다. 빌더, GitOps, 카오스, Prometheus, Loki, Kubernetes 등 외부 시스템 접근을 전부 Python Protocol 인터페이스로 추상화하고, 각 인터페이스에 스텝 구현과 실제 구현을 두어 설정 한 줄로 교체할 수 있게 했다. 이 구조 덕분에 비용이 드는 AWS 클러스터를 켜지 않고도 로컬에서 전체 UI와 로직을 개발하고 검증할 수 있었고, 실제로 상반기 개발 대부분이 스텝 모드에서 이루어졌다. 클러스터는 실배선 검증 시점에만 올렸다.

실시간 상태 갱신은 **WebSocket** 대신 **SSE**를 택했다. 대시보드의 실시간 요구는 서버에서 브라우저로의 단방향 알림뿐이므로 양방향 프로토콜인 **WebSocket**은 과하고, **SSE**는 일반 HTTP 위에서 동작해 별도 인프라 없이 단일 프로세스 구조를 유지할 수 있기 때문이다. 자동화 테스트는 인메모리 DB로 격리 구성해 외부 의존 없이 항상 실행 가능하며, 현재 89건이 통과한다. 대시보드 골격 완성 시점의 19건에서 기능이 추가될 때마다 테스트를 함께 확장해 온 결과다. 저장소는 초기에 SQLite로 구현했으며, 환경 재생성 시 이력 유실 문제를 확인하고 Supabase로의 전환을 확정했다(2장).

카오스 테스트

새 실험 시작

앱에 일부러 장애를 주입하고 AI Agent 가 R 지수를 자동으로 개선해요. 행을 클릭하면 메트릭과 개선 결과를 볼 수 있어요

앱

모든 앱

카오스

모든 종류

상태

전체

총 6건

대상 앱	카오스 종류	시작 시각	기간	R 지수	개선 (Δ)	상태	
comon-be NetworkChaos	NetworkChaos	07/03 15:37	300s	- 목표 0.70	-	실패	>
order-worker StressChaos	StressChaos	07/04 15:37	900s	- 목표 0.70	-	중지됨	>
payment-api PodChaos	PodChaos	07/06 15:37	-	0.81 목표 0.70	0.11	완료	>
online-boutique NetworkChaos	NetworkChaos	07/08 15:37	600s	0.68 목표 0.70	-0.02	완료	>
online-boutique StressChaos	StressChaos	07/10 12:37	300s	0.74 목표 0.70	0.04	완료	>
opus-backend NetworkChaos	NetworkChaos	07/11 15:35	300s	- 목표 0.70	-	주입 중	실행 중지

그림 11은 카오스 테스트 목록으로, 실험별 카오스 종류와 상태(주입 중, 완료, 중지됨, 실패), R
지수와 목표 대비 개선 폭을 한 표로 관리한다. 대시보드에서 카오스 실험의 생성부터 종료까지
전체 생명주기를 제어하는 기능을 구현했다. 새 실험 폼에서 네트워크 지연, Pod 강제 종료,
CPU 부하 3종 중 하나를 선택하고 파라미터를 입력하면, 서버가 허용 범위를 검증한 뒤 Chaos
Mesh CRD를 생성해 클러스터에 주입한다. 주입과 동시에 백그라운드 워치가 붙어 지속 시간
경과와 Chaos Mesh의 회복 상태를 확인하고 CRD를 자동 삭제해 완료 처리하므로, 실험이
끝났는데 장애가 클러스터에 남는 상황이 구조적으로 발생하지 않는다. 실험 상태는 SSE로
실시간 갱신되고, 진행 중 실험의 중지와 동일 앱 중복 실험 차단(409)도 동작한다.

6.5 온프레미스 환경 수행 내용

20

장애처럼 클라우드 환경에서 재현하기 어려운 시나리오를 다룰 수 있다는 점에서 스마트 플러그를 이용한 물리 전원 차단 시나리오도 계획 중이다.

클러스터는 라즈베리파이 **5(3대)**로 구성했다. 이 중 대형 방열팬이 장착된 기종을 마스터 노드로 배치했는데, 마스터는 **API** 서버와 스케줄러를 상시 구동하여 발열 부담이 가장 크기 때문이다. 나머지 **2대**는 워커 노드로 조인했다.

구축 과정에서 가장 큰 걸림돌은 노드 간 통신 자체가 되지 않는 문제였다. 같은 네트워크에 연결된 라즈베리파이끼리 사설 **IP**를 할당받았음에도 상호 **ping**이 달지 않아 **k3s** 노드 조인이 실패했다. 원인을 추적한 결과 통신사 모뎀 공유기가 **DHCP**로 **IP**를 받은 게스트 간 내부 통신을 허용하지 않는 구성이었고, 별도의 공유기를 도입해 **3대**를 동일 **LAN**에 묶는 것으로 해결했다. 교체 후 기기 간 **ping** 통신과 **k3s** 클러스터 조인이 정상 동작함을 확인했으며, 이 네트워크 구성을 이후 실험 환경에 그대로 사용하고 있다.

클러스터 기반 검증을 마친 뒤, 온프레미스 트랙의 다음 단계를 온프레미스 환경에서 동작하는 에이전트 루프 구성으로 확장했다. 선행 연구인 **ChaosEater[4]**의 아키텍처와 메인 코드를 분석하여 카오스 엔지니어링 루프를 전처리, 가설, 실험, 분석, 개선, 보고서의 **6단계**로 정리했고 단계 간 전달 데이터의 스키마 규약을 정의해야 실질적인 역할 분담이 가능하다는 판단에 따라 인터페이스 설계를 함께 진행했다. 이 논의를 바탕으로 **AI** 루프 구현을 양준영과 분담하여, 전처리와 가설 생성, 실험 단계를 온프레미스 파트에서 담당하기로 확정했다.

6.6 AI 루프 수행 내용

카오스 엔지니어링 자동화를 위한 **AI** 루프의 구조를 설계하고, 대시보드 및 클러스터 데이터와의 연동 방향을 구체화했다.

먼저 **ReAct** 방식과 워크플로우형 에이전트 구조를 비교했다. **ReAct**형은 유연하지만 실행마다 경로가 달라져 재현과 디버깅이 어려운 반면, 카오스 엔지니어링은 시스템 분석, 가설 수립, 실험, 결과 분석, 개선의 순서가 명확한 작업이다. 따라서 단계별 입출력이 고정되어 단계 단위로 구현하고 검증할 수 있는 워크플로우형 구조를 채택했으며, 선행 연구인 **ChaosEater[4]**가 루프를 단계형으로 구성한 점도 이 판단을 뒷받침했다.

AI 루프의 입력은 사용자 프롬프트(서비스 특성과 실험 방향)와 **Kubernetes manifest**(배포 구조와 리소스 설정)로 정의했다. 전체 흐름은 시스템 분석, 정상 상태와 장애 가설 정의, 장애 주입 실험, 결과 분석, 개선안 생성, 재실험, 보고서 작성 순이며, 개선이 필요하면 재배포 후 실험 단계로 되돌아간다. 이 흐름은 **3.6절**의 **5개** 역할 에이전트 분담 구조와 맞물리고, 단계 간

전달 데이터의 스키마 규약은 6.5절의 인터페이스 설계에서 함께 확정했다. 반복 횟수는 사용자가 대시보드에서 설정한 상한으로 통제해 실행 시간과 LLM API 비용이 무한히 늘어나는 것을 차단했다.

사용자 개입 방식은 두 안을 비교한 끝에, 에이전트가 시스템 구성을 분석해 실험 후보를 제시하고 사용자는 실행 대상만 선택하는 **Human-in-the-loop**을 채택했다. 전문 지식 부담을 에이전트가 흡수하는 구조라, 인프라 비전문가도 쓸 수 있게 한다는 과제 목표에 부합한다.

에이전트는 툴 호출(**Tool Calling**)로 **kubectl, Prometheus, Loki**를 조회해 실험 전후 비교와 원인 분석, 개선안 생성의 근거를 수집한다. LLM이 생성한 실험안과 개선안도 사람이 만든 실험과 동일한 서버 측 검증 계층을 통과해야 적용되므로, 에이전트 도입이 안전성 기준을 낮추지 않는다.

현재까지 단계 구성, 입력 데이터, 반복 조건, 사용자 개입 방식, 데이터 조회 방식의 정의를 완료하고 개발에 착수했다. 이후 단계별 에이전트와 조회 도구를 구현해 대시보드와 연동하고, 장애 진단 골든셋을 축적해 **RAG**로 에이전트에 제공하는 것까지가 하반기 범위다.

7. 결론 및 기대효과

본 과제는 보고 시점까지 카오스 엔지니어링 자동화의 토대를 실제 동작하는 형태로 완성했다. Terraform 2레이어와 up.sh/down.sh로 실험 환경 전체를 약 20-25분에 구축하고 흔적 없이 파괴하는 자동화를 반복 검증했고, 앱 등록부터 클러스터 내부 빌드, ECR 저장, ArgoCD 동기화, Pod 기동까지 사람 개입 없이 이어지는 **GitOps** 파이프라인을 실서비스급 애플리케이션으로 입증했다. 카오스 주입은 3종 실험의 생성부터 파라미터 검증, 자동 종료까지 전체 생명주기를 대시보드가 제어하며, 온프레미스 트랙에서는 라즈베리파이 기반 k3s 클러스터 구축을 완료했다. 이 모든 결과는 자동화 테스트 89건이 통과하는 상태로 유지되고 있다.

추후에는 이 토대 위에 관측과 지능을 얹어 전 과정 자동화를 완성한다. Prometheus와 Loki를 대시보드에 실배선해 장애 전(baseline)·중(fault)·후(recovery) 구간별 메트릭을 확보하고, 이를

입력으로 **R** 지수를 산출해 실험 이력 간 비교를 제공한다. 이어서 **LangGraph** 워크플로우형 AI 루프를 구현하고, 검증된 장애 사례를 골든셋으로 축적해 **RAG**로 에이전트에 제공함으로써 실험이 반복될수록 진단 품질이 누적 개선되는 구조를 만든다. 마지막으로 온프레미스 **k3s** 클러스터를 대시보드에 연결해 클라우드와 동일한 실험 흐름을 확보한다.

완성된 플랫폼의 기대효과는 세 가지다. 첫째, 카오스 엔지니어링의 진입장벽을 낮춘다. 인프라 전문 지식이 부족한 개발자도 **GitHub** 레포지토리 주소만으로 애플리케이션을 등록하고, 에이전트가 제시한 실험 후보를 선택하는 것만으로 장애 실험과 회복탄력성 진단을 받을 수 있다. 둘째, 회복탄력성을 **R** 지수라는 단일 지표로 정량화하여 개선 전후를 객관적으로 비교하고, 개선의 근거와 효과를 수치로 남긴다. 셋째, 비용 부담 없는 반복 실험이 가능하다. 실험이 필요할 때만 환경을 생성하고 이력은 외부 데이터 저장소에 보존하는 구조라, 환경을 파괴해도 실험 자산이 축적된다.

나아가 본 과제는 클라우드(**EKS**)와 온프레미스(**k3s**)라는 두 인프라 프로파일에서 동일한 장애 시나리오를 실행하고 회복탄력성 특성을 비교하는 실증 자료를 제공한다. 관리형 인프라와 물리 노드는 장애의 양상과 복구 메커니즘이 다르므로, 같은 지표(**R** 지수)로 두 환경을 측정할 비교 데이터는 인프라 선택과 이중화 전략 수립에 참고할 수 있는 결과가 될 것이다. **LLM** 에이전트가 관측 데이터를 근거로 인프라 설정을 자동 개선하는 본 과제의 루프 구조는, 카오스 엔지니어링을 넘어 성능 튜닝이나 비용 최적화 같은 인접 운영 영역으로도 확장 가능한 틀이라는 점에서 졸업과제 이후의 발전 가능성도 열어 둔다.

8. 참고문헌

- [1] X. Zhou et al., "Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study," *IEEE Transactions on Software Engineering*, vol. 47, no. 2, pp. 243–260, 2021. doi: 10.1109/TSE.2018.2887384.
- [2] A. Basiri et al., "Chaos Engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016. doi: 10.1109/MS.2016.60.
- [3] Principles of Chaos Engineering, <https://principlesofchaos.org>
- [4] D. Kikuta et al., "ChaosEater: Fully Automating Chaos Engineering with Large Language

Models," in Proc. IEEE/ACM Int. Conf. Automated Software Engineering (ASE), NIER Track, 2025. arXiv:2501.11107, <https://arxiv.org/abs/2501.11107>

[5] H. Koziolk and N. Eskandani, "Lightweight Kubernetes Distributions: A Performance Comparison of MicroK8s, k3s, k0s, and MicroShift," in Proc. ACM/SPEC Int. Conf. Performance Engineering (ICPE), pp. 17–29, 2023. doi: 10.1145/3578244.3583737.

[6] F. Beetz and S. Harrer, "GitOps: The Evolution of DevOps?," IEEE Software, vol. 39, no. 4, pp. 70–75, 2022. doi: 10.1109/MS.2021.3119106.